

Constraint Satisfaction Problems

A Deep Dive: MEng Course Scheduling at MIT

Based on MIT 6.034 Artificial Intelligence — Slides 3.1.10–3.1.19

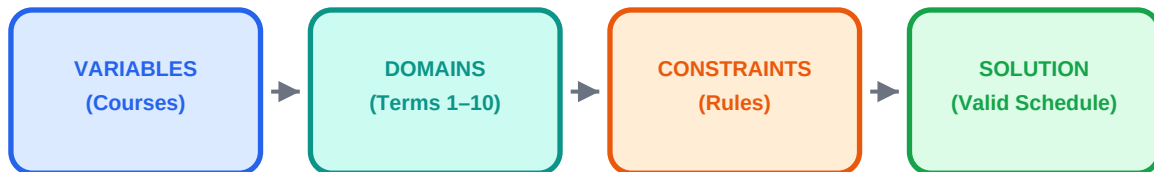


Figure 1 — The four pillars of every CSP: variables, domains, constraints, and a solution.

What you will learn in this document: What a CSP is and why it is useful · How to formulate real scheduling problems as CSPs · Three different formulations of the same problem · How each constraint type is represented · Practical trade-offs between formulations.

1 · What Is a Constraint Satisfaction Problem?

A **Constraint Satisfaction Problem (CSP)** consists of three components:

- **Variables** — a finite set of unknowns, each of which must be assigned a value.
- **Domains** — for every variable, a set of candidate values it may take.
- **Constraints** — logical rules that restrict which combinations of values are acceptable.

The goal is to find an *assignment* — a mapping from every variable to a value in its domain — such that **every constraint is simultaneously satisfied**. If no such assignment exists the problem is said to be *infeasible*.

Key insight: CSPs model *hard* constraints — rules that must be obeyed absolutely. They do not natively model soft preferences. In the scheduling context this means a course cannot be placed in a term that conflicts with its prerequisites, regardless of any other consideration.

1.1 · Why CSPs?

Many real-world problems — scheduling, configuration, planning, map colouring, Sudoku — share this structure. By casting a problem as a CSP we gain access to a rich library of general-purpose algorithms: **backtracking search**, **arc consistency**, **constraint propagation**, and more. Rather than writing a bespoke solver for every new domain, we merely write a CSP formulation.

1.2 · The Constraint Graph

A CSP is naturally visualised as a **constraint graph** in which:

- Each **node** represents a variable.
- Each **edge** (or **hyperedge**) represents a constraint linking two or more variables.

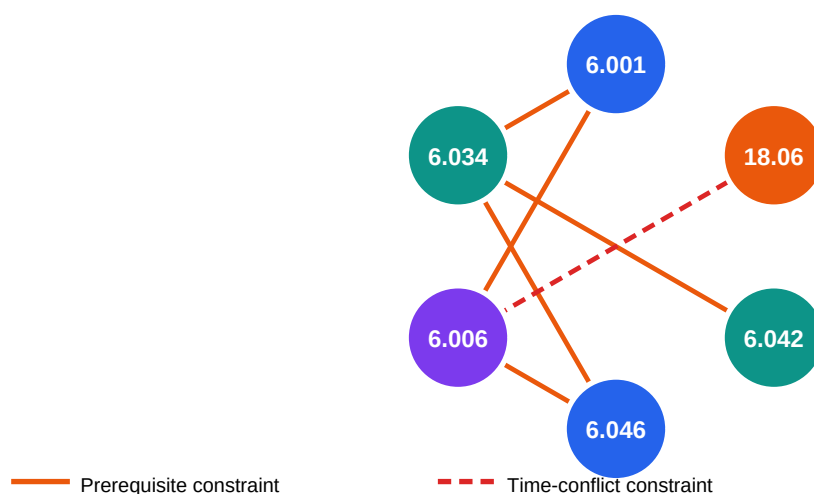


Figure 2 — Example constraint graph for six MIT EECS courses. Solid orange edges denote prerequisite ordering constraints; dashed red edges denote time-conflict constraints.

2 · The MEng Course-Scheduling Problem

MIT EECS students completing the **Master of Engineering (MEng)** degree must schedule approximately **40 courses** across **10 academic terms** (five years, Fall and Spring). For our CSP formulation we assume the complete list of required courses — including all prerequisites — is given.

2.1 · The Four Constraints

The following four rules must be satisfied by any valid schedule:

#	Constraint	Description	Type
C1	Prerequisites	If course B requires course A, then A must appear in an earlier term than B.	Ordering
C2	Term Availability	Some courses are offered only in Fall or only in Spring terms.	Domain Filter
C3	Maximum Load	No more than 4 courses may be taken in any single term.	Cardinality
C4	Time Conflicts	Two courses scheduled at overlapping times cannot be taken in the same term.	Mutual Exclusion

Table 1 — The four hard constraints of the MEng scheduling CSP.

Important: All four constraints are *binary* (or can be decomposed into binary constraints). CSP algorithms work most efficiently when constraints are binary; cardinality constraints (C3) require special handling or an implicit encoding.

3 · Three CSP Formulations

The choice of **variables** and **values** is the most creative step in building a CSP. For the same scheduling problem we can choose very different encodings, each with its own algorithmic consequences.

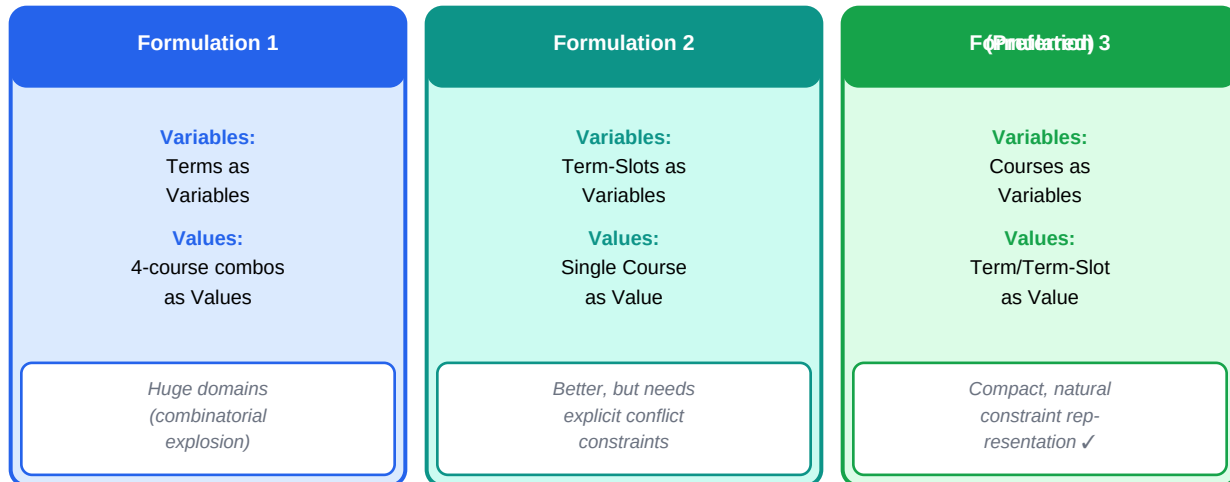


Figure 3 — Side-by-side comparison of the three formulations discussed in slides 3.1.13–3.1.15.

3.1 · Formulation 1 — Terms as Variables

In this formulation we have **10 variables**, one for each academic term. The **value** assigned to a term variable is a *combination* of (up to) four courses that are simultaneously offered and whose meeting times do not conflict.

- **Variables:** $T_1, T_2, \dots, T_{10}$ (the ten terms).
- **Domains:** All feasible 4-course subsets of courses offered in that term type (Fall or Spring).
- **Constraints:** For every prerequisite pair (A, B), the term containing B must have a higher index than the term containing A.

Practical weakness: The domain of each variable contains $C(40, 4) \approx 91\,000$ candidate 4-course combinations before any filtering. This combinatorial explosion makes search algorithms prohibitively slow and is the primary reason this formulation is rarely used.

3.2 · Formulation 2 — Term-Slots as Variables

To avoid the combinatorial blow-up we split each of the 10 terms into four **term-slots**, yielding **40 variables**. Each variable is assigned exactly **one course**. The maximum-load constraint (C3) is now implicit — there are exactly 4 slots per term — but time-conflict and prerequisite constraints must still be stated explicitly.

- **Variables:** 40 term-slot variables ($T_1S_A, T_1S_B, \dots, T_{10}S_D$).
- **Domains:** The set of all courses, filtered by term availability.
- **Constraints:** Prerequisite ordering + time-conflict mutual exclusion within each term.

3.3 · Formulation 3 — Courses as Variables (Preferred)

The most natural formulation for this problem places the **courses themselves** as variables and assigns each course a **term-slot** value. With ~40 courses we have ~40 variables and a compact domain of 40 term-slots.

- **Variables:** One variable per course (e.g. $\text{VAR}_{6.034}$).
- **Domains:** Term-slots in which the course is available (domain filtering handles C2).
- **Constraints:** Prerequisite ordering (C1) + time-conflict mutual exclusion (C4); C3 is handled by limiting the number of variables per term.

Why this formulation is preferred: Domains are small (≤ 40 values), constraints are binary and easy to express, and domain filtering directly removes unavailable terms. This is the formulation developed in detail in Slides 3.1.16–3.1.19.

4 · Formulation 3 in Detail

4.1 · Prerequisite Ordering (Constraint C1)

For every prerequisite relationship — e.g. *6.001 must be taken before 6.034* — we add **two directed binary constraints** between the corresponding variables:

- **term_before**($\text{VAR}_{6.001}$, $\text{VAR}_{6.034}$) — the value assigned to $\text{VAR}_{6.001}$ must be an earlier term than that of $\text{VAR}_{6.034}$.
- **term_after**($\text{VAR}_{6.034}$, $\text{VAR}_{6.001}$) — the complementary constraint on the other arc.

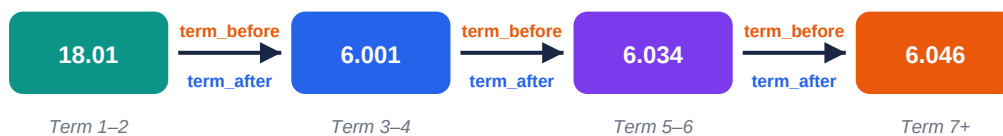


Figure 4 — Directed prerequisite chain: $18.01 \rightarrow 6.001 \rightarrow 6.034 \rightarrow 6.046$. Each arrow carries a *term_before* / *term_after* constraint pair.

4.2 · Term Availability (Constraint C2)

This constraint does **not** appear as an edge in the constraint graph. Instead it is enforced by **domain filtering**: when constructing the domain of a variable we simply remove term-slots that fall in the wrong semester.

Course	Offered In	Domain Restriction
6.034	Fall only	Remove all Spring term-slots from domain
6.830	Spring only	Remove all Fall term-slots from domain
6.006	Fall & Spring	No restriction — all term-slots available
6.046	Fall only	Remove all Spring term-slots from domain

Table 2 — Domain filtering examples for constraint C2 (term availability).

4.3 · Maximum Load (Constraint C3)

By using **term-slots** as values instead of plain terms we encode the load constraint implicitly: if each term has exactly 4 slots and each slot can hold at most one course, the cap of 4 courses per term is guaranteed by construction.

	Fall 1	Spr 1	Fall 2	Spr 2	Fall 3
Slot A	6.001	6.034	6.046	6.009	Thesis
Slot B	6.042	6.006	6.004	6.830	6.UAT
Slot C	18.01	18.06	6.033	—	—
Slot D	8.01	—	—	—	—

Figure 5 — Term × Slot assignment grid. Each cell holds exactly one course. Empty cells (—) indicate unused slots. The grid structure enforces the 4-course-per-term limit.

4.4 · Time Conflicts (Constraint C4)

Two courses that meet at overlapping times cannot be placed in the **same term**. This is captured by a mutual-exclusion constraint between their variables:

- If courses A and B overlap in every term → add constraint: **different_term**(VAR_A, VAR_B).
- If they overlap only in some terms → add a more specific constraint that allows co-scheduling only in non-conflicting terms.

Example: 6.006 (MWF 10–11) and 6.042 (MWF 10–11) always conflict. We add: **different_term**(VAR_{6.006}, VAR_{6.042}). The solver will ensure they appear in different terms.

5 · Solving the CSP — Algorithms Overview

Once the CSP is formulated, a **constraint solver** searches for a satisfying assignment. The two main techniques are:

Algorithm	Idea	When to Use
Backtracking Search	Assign values one variable at a time. If a constraint is violated, undo (backtrack) and try the next value.	Use for small CSPs; efficient with good heuristics.
Arc Consistency (AC-3)	Repeatedly remove values from domains that are provably inconsistent with the constraints.	Pre-processing; improves search; dramatically reduces search space.
Forward Checking	After each assignment, immediately prune domains of unassigned neighbours.	Lightweight propagation; catches failures early.
Min-Conflicts (Local Search)	Start with a complete (possibly invalid) assignment. Iteratively repair the most conflicted constraint.	Very large CSPs where backtracking is too slow.

Table 3 — Common CSP solving algorithms and their use-cases.

5.1 · Variable & Value Ordering Heuristics

The order in which variables are assigned — and values tried — has a profound effect on solver performance. Two classic heuristics are:

- **Minimum Remaining Values (MRV)**: Choose the variable with the fewest legal values remaining in its domain. Intuition: tackle the hardest sub-problem first.
- **Least Constraining Value (LCV)**: When choosing a value for a variable, prefer the value that rules out the fewest choices for neighbouring variables. Intuition: leave as many options open as possible.

For the scheduling CSP: MRV naturally picks the course with the fewest available term-slots (e.g. a course offered in only one specific term), while LCV prefers term-slot assignments that do not block prerequisite successors from being legally scheduled.

6 · Summary & Key Takeaways

Topic	Key Point
CSP Definition	Variables + Domains + Constraints. Goal: find a consistent complete assignment.
Hard Constraints	CSPs enforce rules absolutely — no soft preferences. Either satisfied or not.
Formulation Matters	The choice of variables and values determines algorithmic tractability. Formulation 3 (courses as variables)
Domain Filtering	Availability constraints are best handled by pruning domains, not by adding extra constraint edges.
Implicit Constraints	Term-slots implicitly cap the load at 4 courses per term — no extra constraint needed.
Binary Constraints	Prerequisite and time-conflict constraints are naturally binary and direction-aware (term_before / term_after)
Algorithms	Backtracking + arc consistency + good heuristics (MRV, LCV) solve real scheduling CSPs efficiently.